

Pana ERP v3.0 - Architecture & Implementation Guide

Document Version: 3.0.0

Last Updated: 2025-12-29

Status: Active Development

Previous Version: v1.3 (Cancelled)

Table of Contents

1. Executive Summary

2. Core Philosophy

3. Technology Stack

4. Schema-First Type System

5. Factory Pattern Architecture

6. Directory Structure

7. Component Architecture

8. API Layer

9. State Management

10. Styling System

11. Error Handling

12. Testing Strategy

13. Implementation Roadmap

14. Migration Guide

15. Appendix

1. Executive Summary

1.1 Strategic Decision

Pana ERP v3.0 represents a fundamental architectural shift from "Page-First" development to **"Schema-First"** development. This decision was made to:

- **Eliminate type drift** between Frappe backend and Next.js frontend
- **Reduce boilerplate** by 70%+ through generic factories
- **Enable rapid module development** (target: new module in <2 hours)
- **Ensure runtime type safety** via auto-generated Zod schemas

1.2 Key Mandates

Mandate	Description
Zero Manual Types	All DocType interfaces MUST be generated from Frappe metadata
Factory Pattern	All CRUD operations MUST use generic hooks and route handlers
Single Source of Truth	<code>types/doctype-types.ts</code> is the ONLY source for DocType types

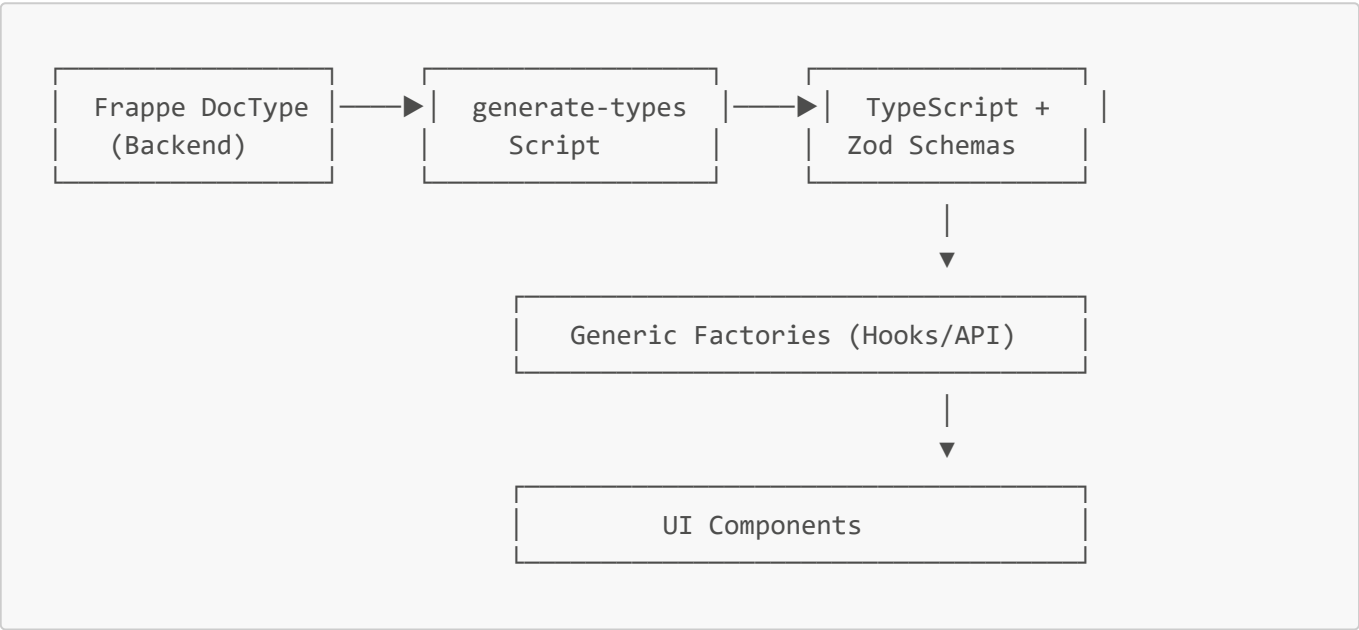
Mandate	Description
Premium UI	Maintain "Big Tech" aesthetic with responsive safety

1.3 Breaking Changes Notice

v3.0 is a clean-slate implementation. All existing modules will break and require migration to the new patterns. This is intentional.

2. Core Philosophy

2.1 Schema-Driven Development



2.2 Type Generation Flow

- 1. **Input:** Frappe DocType name (e.g., "Item", "Customer")
- 2. **Process:** Hit Frappe Metadata API → Parse field definitions
- 3. **Output:**
 - TypeScript interface in `types/doctype-types.ts`
 - Zod schema in `lib/schemas/doctype-schemas.ts`

2.3 Anti-Patterns (DO NOT DO)

```
// ✗ NEVER define manual interfaces for DocTypes
interface Item {
  name: string;
  item_name: string;
  // ... manual fields
}

// ✗ NEVER write custom hooks for standard CRUD
export function useItemsQuery() { ... }
```

```
// ✗ NEVER write custom API routes for standard CRUD
export async function GET(request: Request) {
  // ... custom implementation
}
```

2.4 Correct Patterns (ALWAYS DO)

```
// ☑ Import from generated types
import { Item } from "@types/doctype-types";

// ☑ Use generic hooks
const { data } = useFrappeList<Item>("Item", { filters: [...] });

// ☑ Use factory handlers for API routes
export const GET = createListHandler("Item");
```

3. Technology Stack

3.1 Core Dependencies

Layer	Technology	Version	Purpose
Framework	Next.js	16.x (App Router)	Full-stack React framework
Language	TypeScript	5.9.x (Strict Mode)	Type safety
Styling	Tailwind CSS	v4.x	Utility-first CSS
State	TanStack Query	v5.x	Server state management
Forms	React Hook Form	v7.x	Form handling
Validation	Zod	v3.x	Runtime type validation
Backend	Frappe Framework	v15	REST API provider
Icons	Lucide React	Latest	Consistent iconography
Animations	Framer Motion	v12.x	Micro-interactions

3.2 Dev Dependencies

Tool	Version	Purpose
ESLint	9.x	Code linting
TypeScript ESLint	8.x	TS-specific linting
PostCSS	8.x	CSS processing

3.3 Frappe JS SDK

We use the official `frappe-js-sdk` package for API communication:

```
import { FrappeApp } from "frappe-js-sdk";

const frappe = new FrappeApp(apiUrl, {
  useToken: true,
  token: () => `${apiKey}:${apiSecret}`,
  type: "token",
});
```

4. Schema-First Type System

4.1 Type Generation Script

Location: `scripts/generate-types.js`

Usage:

```
# Generate types for specific DocTypes
pnpm generate-types Item Customer SalesOrder

# Regenerate all known DocTypes
pnpm generate-types --all
```

NPM Script Configuration:

```
{
  "scripts": {
    "generate-types": "node scripts/generate-types.js"
  }
}
```

4.2 Generated Output Files

`types/doctype-types.ts`

```
// AUTO-GENERATED FILE - DO NOT EDIT MANUALLY
// Generated at: 2025-12-29T18:00:00.000Z
// Source: Frappe DocType Metadata API

/**
 * Item DocType
 * @doctype Item
 * @module Stock
```

```

*/
export interface Item {
  /** Primary Key */
  name: string;
  /** Item Code - Unique identifier */
  item_code: string;
  /** Item Name - Display name */
  item_name: string;
  /** Item Group - Category */
  item_group: string;
  /** Stock UOM - Unit of Measure */
  stock_uom: string;
  /** Is Stock Item - 0 or 1 */
  is_stock_item: 0 | 1;
  /** Description */
  description?: string;
  /** Disabled - 0 or 1 */
  disabled?: 0 | 1;
  // ... more fields

  // Standard Frappe fields
  owner?: string;
  creation?: string;
  modified?: string;
  modified_by?: string;
  docstatus?: 0 | 1 | 2;
}

/**
 * Item Create Request
 * Fields required to create a new Item
 */
export type ItemCreateRequest = Pick<Item,
  | "item_code"
  | "item_name"
  | "item_group"
  | "stock_uom"
  | "is_stock_item"
> & Partial<Pick<Item,
  | "description"
  | "disabled"
>>;

/**
 * Item Update Request
 * All fields optional for update
 */
export type ItemUpdateRequest = Partial<Omit<Item, "name" | "creation" |
"owner">>>;

```

```
// AUTO-GENERATED FILE - DO NOT EDIT MANUALLY
// Generated at: 2025-12-29T18:00:00.000Z

import { z } from "zod";

/**
 * Item Zod Schema
 * Use for runtime validation
 */
export const ItemSchema = z.object({
  name: z.string(),
  item_code: z.string().min(1, "Item Code is required"),
  item_name: z.string().min(1, "Item Name is required"),
  item_group: z.string().min(1, "Item Group is required"),
  stock_uom: z.string().min(1, "Stock UOM is required"),
  is_stock_item: z.union([z.literal(0), z.literal(1)]),
  description: z.string().optional(),
  disabled: z.union([z.literal(0), z.literal(1)]).optional(),
  owner: z.string().optional(),
  creation: z.string().optional(),
  modified: z.string().optional(),
  modified_by: z.string().optional(),
  docstatus: z.union([z.literal(0), z.literal(1), z.literal(2)]).optional(),
});

export const ItemCreateSchema = ItemSchema.pick({
  item_code: true,
  item_name: true,
  item_group: true,
  stock_uom: true,
  is_stock_item: true,
}).extend({
  description: z.string().optional(),
  disabled: z.union([z.literal(0), z.literal(1)]).optional(),
});

export const ItemUpdateSchema = ItemSchema.partial().omit({
  name: true,
  creation: true,
  owner: true,
});

export type ItemSchemaType = z.infer<typeof ItemSchema>;
```

4.3 Frappe Field Type Mapping

Frappe Type	TypeScript Type	Zod Schema
Data	string	z.string()
Link	string	z.string()

Frappe Type	TypeScript Type	Zod Schema
Select	<code>string</code> (union of options)	<code>z.enum([...])</code>
Int	<code>number</code>	<code>z.number().int()</code>
Float	<code>number</code>	<code>z.number()</code>
Currency	<code>number</code>	<code>z.number()</code>
Check	<code>0 \ 1</code>	<code>z.union([z.literal(0), z.literal(1)])</code>
Text	<code>string</code>	<code>z.string()</code>
Small Text	<code>string</code>	<code>z.string()</code>
Long Text	<code>string</code>	<code>z.string()</code>
Date	<code>string</code>	<code>z.string()</code> (ISO date)
Datetime	<code>string</code>	<code>z.string()</code> (ISO datetime)
Time	<code>string</code>	<code>z.string()</code>
Table	<code>ChildDocType[]</code>	<code>z.array(ChildSchema)</code>

5. Factory Pattern Architecture

5.1 Generic Hooks (`hooks/generic/`)

`useFrappeList<T>`

Fetches a list of documents with filtering, sorting, and pagination.

```
// hooks/generic/useFrappeList.ts
import { useQuery, UseQueryOptions } from "@tanstack/react-query";

export interface FrappeListOptions {
  fields?: string[];
  filters?: [string, string, unknown][];
  orderBy?: { field: string; order: "asc" | "desc" };
  limit?: number;
  offset?: number;
}

export function useFrappeList<T>(
  doctype: string,
  options?: FrappeListOptions,
  queryOptions?: Omit<UseQueryOptions<T[], Error>, "queryKey" | "queryFn">
) {
  return useQuery({
    queryKey: [doctype, "list", options],
    queryFn: async () => {
      const params = new URLSearchParams();
```

```

    if (options?.fields) params.set("fields", JSON.stringify(options.fields));
    if (options?.filters) params.set("filters",
JSON.stringify(options.filters));
    if (options?.orderBy) params.set("order_by", `${options.orderBy.field}
${options.orderBy.order}`);
    if (options?.limit) params.set("limit", String(options.limit));
    if (options?.offset) params.set("offset", String(options.offset));

    const response = await fetch(`/api/${doctype.toLowerCase()}?${params}`);
    if (!response.ok) throw new Error("Failed to fetch");
    const json = await response.json();
    return json.data as T[];
  },
  ...queryOptions,
});
}

```

useFrappeDoc<T>

Fetches a single document by name.

```

// hooks/generic/useFrappeDoc.ts
export function useFrappeDoc<T>({
  doctype: string,
  name: string,
  queryOptions?: Omit<UseQueryOptions<T, Error>, "queryKey" | "queryFn">
}) {
  return useQuery({
    queryKey: [doctype, "doc", name],
    queryFn: async () => {
      const response = await
fetch(`/api/${doctype.toLowerCase()}/${encodeURIComponent(name)}`);
      if (!response.ok) throw new Error("Failed to fetch document");
      const json = await response.json();
      return json.data as T;
    },
    enabled: !!name,
    ...queryOptions,
  });
}

```

useFrappeMutation<T>

Generic mutation hook for create/update/delete operations.

```

// hooks/generic/useFrappeMutation.ts
export function useFrappeMutation<TData, TVariables>({
  doctype: string,
  operation: "create" | "update" | "delete",

```

```

    options?: {
      onSuccess?: (data: TData) => void;
      onError?: (error: Error) => void;
      invalidateQueries?: string[][];
    }
  ) {
    const queryClient = useQueryClient();

    return useMutation({
      mutationFn: async (variables: TVariables) => {
        const method = operation === "create" ? "POST" : operation === "update" ?
"PUT" : "DELETE";
        const response = await fetch(`/api/${doctype.toLowerCase()}`, {
          method,
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify(variables),
        });
        if (!response.ok) {
          const error = await response.json();
          throw new Error(error.details || "Operation failed");
        }
        return response.json() as Promise<TData>;
      },
      onSuccess: (data) => {
        // Invalidate related queries
        queryClient.invalidateQueries({ queryKey: [doctype] });
        options?.invalidateQueries?.forEach(key => {
          queryClient.invalidateQueries({ queryKey: key });
        });
        options?.onSuccess?.(data);
      },
      onError: options?.onError,
    });
  }
}

```

5.2 API Factory (`lib/api-factory.ts`)

Factory functions that generate Next.js API route handlers.

```

// lib/api-factory.ts
import { NextRequest, NextResponse } from "next/server";
import { frappeClient } from "../frappe-client";
import { ZodSchema } from "zod";

interface ListHandlerOptions {
  allowedFields?: string[];
  defaultFilters?: [string, string, unknown][];
  defaultSort?: { field: string; order: "asc" | "desc" };
  defaultLimit?: number;
}

```

```

export function createListHandler(doctype: string, options?: ListHandlerOptions) {
  return async function GET(request: NextRequest) {
    try {
      const { searchParams } = new URL(request.url);

      // Parse query parameters
      const fields = searchParams.get("fields")
        ? JSON.parse(searchParams.get("fields")!)
        : options?.allowedFields || ["*"];
      const filters = searchParams.get("filters")
        ? JSON.parse(searchParams.get("filters")!)
        : options?.defaultFilters || [];
      const orderBy = searchParams.get("order_by")
        || `${options?.defaultSort?.field || "creation"}
${options?.defaultSort?.order || "desc"}`;
      const limit = parseInt(searchParams.get("limit") ||
String(options?.defaultLimit || 100));
      const offset = parseInt(searchParams.get("offset") || "0");

      const data = await frappeClient.db.getDocList(doctype, {
        fields,
        filters,
        orderBy: { field: orderBy.split(" ")[0], order: orderBy.split(" ")[1] as
"asc" | "desc" },
        limit,
        start: offset,
      });

      return NextResponse.json({ success: true, data });
    } catch (error) {
      return NextResponse.json(frappeClient.handleError(error), { status: 500 });
    }
  };
}

export function createGetHandler(doctype: string) {
  return async function GET(
    request: NextRequest,
    { params }: { params: Promise<{ name: string }> }
  ) {
    try {
      const { name } = await params;
      const data = await frappeClient.db.getDoc(doctype, name);
      return NextResponse.json({ success: true, data });
    } catch (error) {
      return NextResponse.json(frappeClient.handleError(error), { status: 500 });
    }
  };
}

export function createCreateHandler<T>(doctype: string, schema?: ZodSchema<T>) {
  return async function POST(request: NextRequest) {
    try {
      const body = await request.json();

```

```
// Validate with Zod if schema provided
if (schema) {
  const result = schema.safeParse(body);
  if (!result.success) {
    return NextResponse.json({
      success: false,
      error: "Validation Error",
      details: result.error.flatten().fieldErrors,
    }, { status: 400 });
  }
}

const data = await frappeClient.db.createDoc(doctype, body);
return NextResponse.json({ success: true, data }, { status: 201 });
} catch (error) {
  return NextResponse.json(frappeClient.handleError(error), { status: 500 });
}
};
}

export function createUpdateHandler<T>(doctype: string, schema?: ZodSchema<T>) {
  return async function PUT(request: NextRequest) {
    try {
      const { searchParams } = new URL(request.url);
      const name = searchParams.get("name");

      if (!name) {
        return NextResponse.json({
          success: false,
          error: "Missing Parameter",
          details: "Document name is required",
        }, { status: 400 });
      }

      const body = await request.json();

      if (schema) {
        const result = schema.safeParse(body);
        if (!result.success) {
          return NextResponse.json({
            success: false,
            error: "Validation Error",
            details: result.error.flatten().fieldErrors,
          }, { status: 400 });
        }
      }

      const data = await frappeClient.db.updateDoc(doctype, name, body);
      return NextResponse.json({ success: true, data });
    } catch (error) {
      return NextResponse.json(frappeClient.handleError(error), { status: 500 });
    }
  };
}
```

```

}

export function createDeleteHandler(doctype: string) {
  return async function DELETE(request: NextRequest) {
    try {
      const { searchParams } = new URL(request.url);
      const name = searchParams.get("name");

      if (!name) {
        return NextResponse.json({
          success: false,
          error: "Missing Parameter",
          details: "Document name is required",
        }, { status: 400 });
      }

      await frappeClient.db.deleteDoc(doctype, name);
      return NextResponse.json({ success: true, message: "Document deleted" });
    } catch (error) {
      return NextResponse.json(frappeClient.handleError(error), { status: 500 });
    }
  };
}

```

5.3 Usage Example

```

// app/api/stock/item/route.ts
import { createListHandler, createCreateHandler } from "@/lib/api-factory";
import { ItemCreateSchema } from "@/lib/schemas/doctype-schemas";

export const GET = createListHandler("Item", {
  allowedFields: ["name", "item_code", "item_name", "item_group", "stock_uom", "disabled"],
  defaultSort: { field: "creation", order: "desc" },
});

export const POST = createCreateHandler("Item", ItemCreateSchema);

```

6. Directory Structure

6.1 Complete Project Structure

```

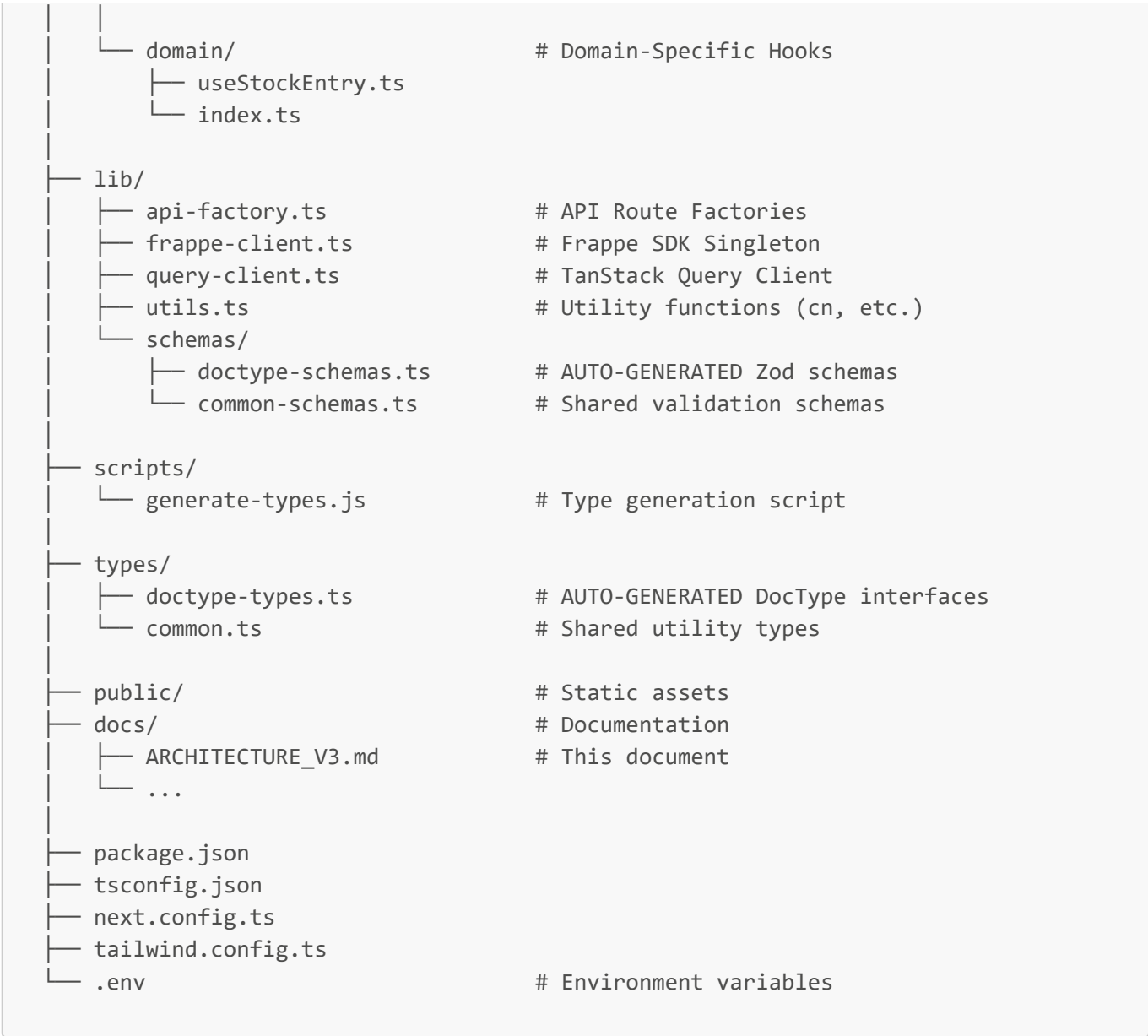
pana-erp/
├── app/
│   ├── api/                                # API Routes
│   │   ├── stock/
│   │   │   ├── item/
│   │   │   │   └── route.ts                # GET (list), POST (create)

```

```

├── [name]/
│   └── route.ts # GET (single), PUT, DELETE
│   └── ...
├── ...
├── stock/ # Stock Module Pages
│   ├── item/
│   │   ├── page.tsx # Items List
│   │   ├── new/
│   │   │   └── page.tsx # Create Item
│   │   └── [name]/
│   │       ├── page.tsx # View Item
│   │       └── edit/
│   │           └── page.tsx # Edit Item
│   └── ...
├── layout.tsx # Root Layout
├── globals.css # Global Styles
├── page.tsx # Dashboard
├── components/
│   ├── core/ # Primitive UI Components
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── select.tsx
│   │   ├── card.tsx
│   │   ├── dialog.tsx
│   │   ├── dropdown-menu.tsx
│   │   ├── separator.tsx
│   │   ├── skeleton.tsx
│   │   ├── tooltip.tsx
│   │   └── index.ts # Barrel export
│   ├── layout/ # Layout Components
│   │   ├── app-provider.tsx # Global providers
│   │   ├── app-sidebar.tsx # Main sidebar
│   │   ├── sidebar-shell.tsx # Shell with sidebar
│   │   └── index.ts
│   └── smart/ # Frappe-Aware Components
│       ├── page-header.tsx # Floating header
│       ├── status-badge.tsx # Auto-colored status
│       ├── frappe-select.tsx # Async Frappe dropdown
│       ├── data-field.tsx # Label + Input + Error
│       ├── data-table.tsx # Generic data table
│       ├── empty-state.tsx # No data display
│       ├── loading-state.tsx # Loading skeleton
│       └── index.ts
├── hooks/
│   ├── generic/ # Generic Frappe Hooks
│   │   ├── useFrappelist.ts
│   │   ├── useFrappeDoc.ts
│   │   ├── useFrappeMutation.ts
│   │   ├── useFrappeOptions.ts # For Link field dropdowns
│   │   └── index.ts

```



6.2 Directory Responsibilities

Directory	Purpose	Imports From
components/core/	Primitive UI (no business logic)	Only lib/utils
components/layout/	Page structure, providers	core/, lib/
components/smart/	Frappe-aware, business logic	core/, hooks/, types/
hooks/generic/	Reusable Frappe hooks	lib/, types/
hooks/domain/	Complex business logic	generic/, types/
lib/	Utilities, clients, factories	Nothing (leaf)
types/	TypeScript definitions	Nothing (leaf)

7. Component Architecture

7.1 Core Components (components/core/)

These are pure UI primitives with no business logic. Based on shadcn/ui patterns.

```
// components/core/button.tsx
import { cva, type VariantProps } from "class-variance-authority";

const buttonVariants = cva(
  "inline-flex items-center justify-center rounded-xl text-sm font-semibold transition-all duration-200 focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring disabled:opacity-50 disabled:pointer-events-none",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground shadow-lg shadow-primary/20 hover:shadow-primary/30 hover:-translate-y-0.5",
        destructive: "bg-destructive text-destructive-foreground hover:bg-destructive/90",
        outline: "border border-input bg-background hover:bg-accent hover:text-accent-foreground",
        secondary: "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-11 px-6",
        sm: "h-9 px-4",
        lg: "h-12 px-8",
        icon: "h-10 w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);
```

7.2 Smart Components ([components/smart/](#))

PageHeader (Refactored with Responsive Fix)

```
// components/smart/page-header.tsx
"use client";

import { Button } from "@components/core/button";
import { ArrowLeft, Loader2 } from "lucide-react";
import { useRouter } from "next/navigation";
import { cn } from "@lib/utils";
import React from "react";

interface PageHeaderProps {
```

```

    backUrl?: string;
    onBack?: () => void;
    label?: string;
    title: string;
    status?: {
      label: string;
      variant: "success" | "warning" | "destructive" | "default";
    };
    hasChanges?: boolean;
    primaryAction?: {
      label: string;
      onClick: () => void;
      icon?: React.ReactNode;
      loading?: boolean;
      disabled?: boolean;
    };
    children?: React.ReactNode;
    className?: string;
  }

  const statusVariants = {
    success: "bg-emerald-500/10 text-emerald-600 border-emerald-500/20",
    warning: "bg-amber-500/10 text-amber-600 border-amber-500/20",
    destructive: "bg-destructive/10 text-destructive border-destructive/20",
    default: "bg-secondary text-muted-foreground border-border",
  };

  export function PageHeader({
    backUrl,
    onBack,
    label,
    title,
    status,
    hasChanges,
    primaryAction,
    children,
    className,
  }: PageHeaderProps) {
    const router = useRouter();

    const handleBack = () => {
      if (onBack) onBack();
      else if (backUrl) router.push(backUrl);
      else router.back();
    };

    return (
      <div
        className={cn(
          // Base styles
          "flex items-center justify-between sticky top-0 z-20",
          "bg-white/80 backdrop-blur-xl rounded-full shadow-sm border border-
white/40",
          "p-2 pr-4",

```

```

        "animate-in fade-in slide-in-from-top-2 duration-500",
        className
    })
}
>
{/* Left section - min-w-0 prevents overflow */}
<div className="flex items-center gap-4 min-w-0 flex-1">
    <Button
        variant="ghost"
        size="icon"
        onClick={handleBack}
        className="rounded-full h-10 w-10 hover:bg-white shrink-0"
    >
        <ArrowLeft className="h-5 w-5 text-muted-foreground" />
    </Button>

    <div className="h-8 w-[1px] bg-border/50 shrink-0 hidden sm:block" />

    {/* Title container with truncate */}
    <div className="flex flex-col min-w-0">
        {label && (
            <span className="text-xs font-bold text-muted-foreground uppercase tracking-wider truncate">
                {label}
            </span>
        )}
        <h1 className="text-lg font-bold leading-none truncate">{title}</h1>
    </div>

    {/* Status badge - hidden on mobile */}
    {status && (
        <div
            className={cn(
                "px-3 py-1 rounded-full text-xs font-bold uppercase tracking-wide border shrink-0",
                "hidden md:block",
                statusVariants[status.variant]
            )}
        >
            {status.label}
        </div>
    )}

    {/* Unsaved indicator - hidden on mobile */}
    {hasChanges && (
        <div className="hidden sm:flex items-center gap-2 px-3 py-1.5 bg-amber-500/10 rounded-full shrink-0">
            <div className="h-2 w-2 rounded-full bg-amber-500 animate-pulse" />
            <span className="text-xs font-semibold text-amber-600">Unsaved</span>
        </div>
    )}
</div>

{/* Right section - shrink-0 prevents squishing */}
<div className="flex items-center gap-2 shrink-0">

```

```

    {children}

    {primaryAction && (
      <Button
        onClick={primaryAction.onClick}
        disabled={primaryAction.disabled || primaryAction.loading}
        className="rounded-full px-6"
      >
        {primaryAction.loading ? (
          <Loader2 className="h-4 w-4 mr-2 animate-spin" />
        ) : (
          primaryAction.icon && <span className="mr-2">{primaryAction.icon}
</span>
        )}
        <span className="hidden sm:inline">{primaryAction.label}</span>
        <span className="sm:hidden">{primaryAction.icon ? "" :
primaryAction.label}</span>
      </Button>
    )}
  </div>
</div>
);
}

```

StatusBadge

```

// components/smart/status-badge.tsx
import { cn } from "@lib/utils";

const statusConfig = {
  // Document Status
  draft: { label: "Draft", className: "bg-zinc-100 text-zinc-600 border-zinc-200" },
  submitted: { label: "Submitted", className: "bg-blue-50 text-blue-600 border-blue-200" },
  cancelled: { label: "Cancelled", className: "bg-red-50 text-red-600 border-red-200" },

  // Custom Status
  active: { label: "Active", className: "bg-emerald-50 text-emerald-600 border-emerald-200" },
  disabled: { label: "Disabled", className: "bg-zinc-100 text-zinc-500 border-zinc-200" },
  pending: { label: "Pending", className: "bg-amber-50 text-amber-600 border-amber-200" },
  completed: { label: "Completed", className: "bg-green-50 text-green-600 border-green-200" },
};

type StatusType = keyof typeof statusConfig;

```

```

export function StatusBadge({ status, className }: { status: StatusType | string;
className?: string }) {
  const config = statusConfig[status as StatusType] || {
    label: status,
    className: "bg-gray-100 text-gray-600 border-gray-200",
  };

  return (
    <span
      className={cn(
        "inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium
border",
        config.className,
        className
      )}
    >
      {config.label}
    </span>
  );
}

```

FrappeSelect

```

// components/smart/frappe-select.tsx
"use client";

import { useFrappeList } from "@/hooks/generic";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from
"@/components/core/select";
import { Loader2 } from "lucide-react";

interface FrappeSelectProps {
  doctype: string;
  value?: string;
  onChange: (value: string) => void;
  placeholder?: string;
  labelField?: string;
  valueField?: string;
  filters?: [string, string, unknown][];
  disabled?: boolean;
}

export function FrappeSelect({
  doctype,
  value,
  onChange,
  placeholder = "Select...",
  labelField = "name",
  valueField = "name",
  filters,
  disabled,

```

```

}): FrappeSelectProps) {
  const { data, isLoading } = useFrappeList<Record<string, string>>(doctype, {
    fields: [valueField, labelField],
    filters,
    limit: 500,
  });

  return (
    <Select value={value} onChange={onChange} disabled={disabled ||
    isLoading}>
      <SelectTrigger>
        {isLoading ? (
          <Loader2 className="h-4 w-4 animate-spin" />
        ) : (
          <SelectValue placeholder={placeholder} />
        )}
      </SelectTrigger>
      <SelectContent>
        {data?.map((item) => (
          <SelectItem key={item[valueField]} value={item[valueField]}>
            {item[labelField]}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  );
}

```

8. API Layer

8.1 API Route Contract

All API routes return consistent response shapes:

Success Response

```

{
  success: true,
  data: T, // The payload
  message?: string // Optional success message
}

```

Error Response

```

{
  success: false,
  error: string, // Error type (e.g., "Validation Error")
}

```

```

    details: string | object,    // Human-readable details or field errors
    statusCode?: number          // HTTP status code
  }

```

8.2 API Route Structure

```

/api/{module}/{doctype}
  GET    → List all documents (with filters)
  POST   → Create new document

/api/{module}/{doctype}/{name}
  GET    → Get single document
  PUT    → Update document
  DELETE → Delete document

/api/{module}/{doctype}/options
  GET    → Get dropdown options (Link fields)

```

8.3 Example Implementation

```

// app/api/stock/item/route.ts
import { createListHandler, createCreateHandler } from "@lib/api-factory";
import { ItemCreateSchema } from "@lib/schemas/doctype-schemas";

export const GET = createListHandler("Item", {
  allowedFields: ["name", "item_code", "item_name", "item_group", "stock_uom",
"is_stock_item", "disabled", "creation"],
  defaultSort: { field: "creation", order: "desc" },
  defaultLimit: 100,
});

export const POST = createCreateHandler("Item", ItemCreateSchema);

```

```

// app/api/stock/item/[name]/route.ts
import { createGetHandler, createUpdateHandler, createDeleteHandler } from
"@lib/api-factory";
import { ItemUpdateSchema } from "@lib/schemas/doctype-schemas";

export const GET = createGetHandler("Item");
export const PUT = createUpdateHandler("Item", ItemUpdateSchema);
export const DELETE = createDeleteHandler("Item");

```

9. State Management

9.1 TanStack Query Configuration

```
// lib/query-client.ts
import { QueryClient } from "@tanstack/react-query";

export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000, // 1 minute
      gcTime: 5 * 60 * 1000, // 5 minutes (formerly cacheTime)
      retry: 1,
      refetchOnWindowFocus: false,
    },
    mutations: {
      retry: 0,
    },
  },
});
```

9.2 Query Key Convention

```
// Query keys follow [doctype, operation, ...params] pattern
[doctype, "list", options] // List query
[doctype, "doc", name] // Single document
[doctype, "options"] // Dropdown options
[doctype, "search", searchTerm] // Search results
```

9.3 Optimistic Updates

```
// Example: Optimistic delete
const deleteMutation = useFrappeMutation<void, string>("Item", "delete", {
  onMutate: async (name) => {
    await queryClient.cancelQueries({ queryKey: ["Item", "list"] });
    const previous = queryClient.getQueryData<Item[]>(["Item", "list"]);
    queryClient.setQueryData<Item[]>(["Item", "list"], (old) =>
      old?.filter((item) => item.name !== name)
    );
    return { previous };
  },
  onError: (err, name, context) => {
    queryClient.setQueryData(["Item", "list"], context?.previous);
  },
});
```

10. Styling System

10.1 Design Tokens

All tokens are defined in `app/globals.css` using `oklch` color space for perceptual uniformity.

```
@theme inline {
  /* Font */
  --font-sans: var(--font-jakarta), system-ui, -apple-system, sans-serif;

  /* Colors - OKLCH for perceptual uniformity */
  --color-background: oklch(0.98 0.01 240);
  --color-foreground: oklch(0.15 0.02 240);
  --color-primary: oklch(0.15 0.02 240);
  --color-primary-foreground: oklch(1 0 0);
  --color-secondary: oklch(0.96 0.01 240);
  --color-muted: oklch(0.96 0.01 240);
  --color-muted-foreground: oklch(0.55 0.02 240);
  --color-destructive: oklch(0.6 0.15 25);
  --color-border: oklch(0.92 0.01 240);

  /* Radius - Premium curved aesthetic */
  --radius-sm: 0.5rem;
  --radius-md: 0.75rem;
  --radius-lg: 1rem;
  --radius-xl: 1.5rem;

  /* Shadows - Atmospheric depth */
  --shadow-sm: 0 4px 6px -1px rgb(0 0 0 / 0.02);
  --shadow-md: 0 12px 24px -8px rgb(0 0 0 / 0.04);
  --shadow-lg: 0 20px 32px -8px rgb(0 0 0 / 0.06);
}
```

10.2 Utility Classes

```
/* Glassmorphism */
.glass {
  background: rgba(255, 255, 255, 0.75);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.4);
}

/* Hover Effects */
.hover-lift:hover {
  transform: translateY(-0.25rem);
  box-shadow: var(--shadow-lg);
}

/* Skeleton Loading */
.skeleton {
  background: linear-gradient(90deg, var(--color-secondary) 25%, oklch(0.93 0.01 240) 50%, var(--color-secondary) 75%);
}
```

```
background-size: 200% 100%;
animation: shimmer 1.5s ease-in-out infinite;
}
```

11. Error Handling

11.1 Frappe Error Parsing

The `frappe-client.ts` includes intelligent error parsing:

```
public handleError(error: unknown): ApiErrorResponse {
  if (isFrappeError(error)) {
    // Extract user-friendly message from _server_messages
    if (error._server_messages) {
      const serverMessages = JSON.parse(error._server_messages);
      // ... parse and return friendly message
    }

    // Infer from exception type
    if (rawError.includes("DuplicateEntryError")) {
      return { details: "A record with these details already exists." };
    }
    // ... more mappings
  }
}
```

11.2 Client-Side Error Display

Use `sonner` toast for user feedback:

```
import { toast } from "sonner";

// In mutation onError
onError: (error: Error) => {
  toast.error(error.message, {
    description: "Please try again or contact support.",
    action: {
      label: "Retry",
      onClick: () => mutation.mutate(lastVariables),
    },
  });
}
```

12. Testing Strategy

12.1 Type Generation Testing

```
# Test type generation
node scripts/generate-types.js Item --dry-run

# Verify output
cat types/doctype-types.ts
```

12.2 API Factory Testing

```
# Test API endpoints
curl http://localhost:3000/api/stock/item
curl http://localhost:3000/api/stock/item/ITEM-001
```

12.3 Integration Testing

```
// __tests__/api/item.test.ts
describe("Item API", () => {
  it("should list items", async () => {
    const res = await fetch("/api/stock/item");
    expect(res.ok).toBe(true);
    const json = await res.json();
    expect(json.success).toBe(true);
    expect(Array.isArray(json.data)).toBe(true);
  });
});
```

13. Implementation Roadmap

Phase 1: Foundation (Current)

Task	Status	Priority
Create <code>scripts/generate-types.js</code>	 In Progress	P0
Generate <code>types/doctype-types.ts</code>	 Pending	P0
Generate <code>lib/schemas/doctype-schemas.ts</code>	 Pending	P0
Create <code>lib/api-factory.ts</code>	 Pending	P1
Create <code>hooks/generic/</code>	 Pending	P1

Phase 2: Directory Restructure

Task	Status	Priority
Create <code>components/core/</code>	 Pending	P1

Task	Status	Priority
Create <code>components/layout/</code>	Pending	P1
Create <code>components/smart/</code>	Pending	P1
Migrate existing components	Pending	P1
Update all imports	Pending	P1

Phase 3: Item Module Migration

Task	Status	Priority
Rewrite <code>app/api/stock/item</code> routes	Pending	P1
Rewrite <code>app/stock/item</code> pages	Pending	P1
Fix PageHeader responsive issues	Pending	P2
Test full CRUD flow	Pending	P1

Phase 4: Rollout (Future)

- Apply patterns to Customer, Sales Order, etc.
- Migrate all modules to v3.0 architecture

14. Migration Guide

14.1 For Existing Hooks

```
// BEFORE (v1.3)
import { useItemsQuery } from "@/hooks/data/useItemsQuery";
const { data } = useItemsQuery({ group: "Raw Materials" });

// AFTER (v3.0)
import { useFrappelist } from "@/hooks/generic";
import { Item } from "@/types/doctype-types";
const { data } = useFrappelist<Item>("Item", {
  filters: [{"item_group", "=", "Raw Materials"}],
});
```

14.2 For Existing Types

```
// BEFORE (v1.3)
import { Item } from "@/types/item";

// AFTER (v3.0)
import { Item } from "@/types/doctype-types";
```

14.3 For Existing API Routes

```
// BEFORE (v1.3) - 50+ lines of custom logic
export async function GET(request: Request) {
  try {
    const items = await frappeClient.db.getDocList("Item", { ... });
    return NextResponse.json({ success: true, data: { items } });
  } catch (error) {
    // ... error handling
  }
}

// AFTER (v3.0) - 3 lines
import { createListHandler } from "@lib/api-factory";
export const GET = createListHandler("Item");
```

15. Appendix

15.1 Environment Variables

```
# Frappe API Configuration
NEXT_PUBLIC_ERP_API_URL=https://your-frappe-instance.com
ERP_API_KEY=your_api_key
ERP_API_SECRET=your_api_secret
```

15.2 NPM Scripts

```
{
  "scripts": {
    "dev": "next dev --turbopack",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "generate-types": "node scripts/generate-types.js"
  }
}
```

15.3 Frappe API Reference

Endpoint	Method	Purpose
/api/resource/{doctype}	GET	List documents
/api/resource/{doctype}	POST	Create document

Endpoint	Method	Purpose
/api/resource/{doctype}/{name}	GET	Get document
/api/resource/{doctype}/{name}	PUT	Update document
/api/resource/{doctype}/{name}	DELETE	Delete document
/api/resource/DocType/{name}	GET	Get DocType metadata

End of Document

This architecture document is the single source of truth for Pana ERP v3.0 development.